Reviewer Pack — Symmetric Tensor Rank Gap in Permanent vs Determinant Decomp…

Entry 8de14a1366b2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 01:23:04 UTC

Symmetric Tensor Rank Gap in Permanent vs Determinant Decompositions

Entry ID: `8de14a1366b2` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-10 01:23:04 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory of Symmetric Groups **Field B** (complexity object): Monotone Circuit Size for 3-CNFs

Statement:

> For a random 3-CNF formula Φ with n variables, the dimension of the irreducible constituent λ in the symmetric square of the permanent's representation (computed via Young diagrams) exceeds the dimension of the corresponding constituent in the determinant's representation by a factor of Ω(2^{n/4}) with probability ≥ 0.95 over Φ.

Rationale (proposer's reasoning):

> Symmetric tensor decompositions reveal structural asymmetry between permanent and determinant. The Young diagram dimensions capture combinatorial complexity, and their exponential gap could imply inherent circuit size separation via representation-theoretic obstructions.

Taxonomy category: `GCT_DET_PERM` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `2ba31ecdad673661`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM₁	LLM₂
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
from math import factorial

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def hook_length_formula(n, k):
    numerator = 1
    denominator = 1
    for i in range(1, n + 1):
        for j in range(1, k + 1):
            numerator *= (n - i + j)
            denominator *= j
    return numerator // denominator

def permanent(n, matrix):
    if n == 0:
        return 1
    if n == 1:
        return matrix[0][0]
    det = 0
    for j in range(n):
        submatrix = [[matrix[i][k] for k in range(j) + range(j+1, n)] for i in range(1, n)]
        det += ((-1) ** j) * matrix[0][j] * permanent(n - 1, submatrix)
    return det

def determinant(n, matrix):
    if n == 0:
        return 1
    if n == 1:
        return matrix[0][0]
    det = 0
    for j in range(n):
```

```
        submatrix = [[matrix[i][k] for k in range(j) + range(j+1, n)] for i in range(1, n)]
        det += ((-1) ** j) * matrix[0][j] * determinant(n - 1, submatrix)
    return det

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 16
    num_instances = 30
    count_supporting = 0
    counterexample = ""

    for _ in range(num_instances):
        # Generate a random 3-CNF formula with n variables
        clauses = []
        for _ in range(2 * n):
            literals = [random.randint(1, n), random.randint(-n, -1)]
            random.shuffle(literals)
            clause = (literals[0], literals[1])
            clauses.append(clause)

        # Compute the permanent and determinant representations
        perm_matrix = [[0] * n for _ in range(n)]
        det_matrix = [[0] * n for _ in range(n)]

        for i in range(n):
            for j in range(n):
                if (i + 1, j + 1) in clauses or (-i - 1, -j - 1) in clauses:
                    perm_matrix[i][j] = 1
                    det_matrix[i][j] = 1

        perm_val = permanent(n, perm_matrix)
        det_val = determinant(n, det_matrix)

        # Compute the ratio of dominant irreducible component dimensions
        perm_dim = hook_length_formula(n, n)
        det_dim = hook_length_formula(n, n - 1)
        ratio = perm_dim / det_dim

        if ratio >= 2 ** (n / 4) * 0.95:
            count_supporting += 1
        else:
            counterexample = f"Ratio {ratio} < 2^(n/4) * 0.95"

    return {
        "metric_name": "Ratio of Permanent to Determinant Dimensions",
        "metric_value": ratio,
        "instances_tested": num_instances,
        "conjecture_holds": count_supporting >= 0.8 * num_instances,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = list(map(int, sys.argv[1:])) if sys.argv[1:] else [2**i + 1 for i in range(5, 30)]

    results = []
```

```
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_ratio = sum(r["metric_value"] for r in results) / len(results)
    std_ratio = (sum((r["metric_value"] - mean_ratio) ** 2 for r in results) / len(results)) ** 0.
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_ratio} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6dfe40e5.py", line 110, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6dfe40e5.py", line 83, in run_trial
    perm_val = permanent(n, perm_matrix)
               ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6dfe40e5.py", line 42, in permanent
    submatrix = [[matrix[i][k] for k in range(j) + range(j+1, n)] for i in range(1, n)]
                                      ~~~~~~~~~~^~~~~~~~~~~~~~~
TypeError: unsupported operand type(s) for +: 'range' and 'range'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed due to Python TypeError in range concatenation, preventing data collection. Support condition (80% success rate) cannot be evaluated without valid results. | next: Fix the range concatenation error in the permanent function by using list concatenation (e.g., list(range(j)) + list(range(j+1, n)))

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111849
2	propose	ollama_remote	qwen3:8b	0	0	81960
3	propose	ollama_remote	qwen3:8b	0	0	56150
4	preregistration	ollama_remote	qwen3:8b	0	0	24181
5	novelty	ollama_remote	qwen3:8b	0	0	20790
6	novelty	ollama_remote	qwen3:8b	0	0	13949
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10002
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12130
9	judge	ollama_remote	qwen3:8b	0	0	19770

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 350780 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/8de14a1366b2.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/8de14a1366b2.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/8de14a1366b2.tar.gz` (if generated)